

中山大學
SUN YAT-SEN UNIVERSITY

计算机图形学 | 作业一

基于 OpenGL 的字母绘制实验

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

指 导 教 师: 陶钧

起 始 日 期: 2025 年 10 月 20 日

结 束 日 期: 2025 年 11 月 7 日

实 验 地 点: B402

目录

1 实验概述 **3**
1.1 实验目的 . . . 3
1.2 实验内容 . . . 3

2 环境搭建 **4**

3 二维字母绘制 **7**
3.1 字母选取规则 . . . 7
3.2 绘制思路 . . . 7
3.3 核心代码实现 . . . 7
3.4 绘制结果 . . . 9
3.5 讨论一：不同绘制方式的开销比较 . . . 9
3.5.1 图元类型分析 . . . 9
3.5.2 具体开销计算 . . . 9
3.5.3 结果分析 . . . 10
3.6 讨论二：投影方式比较 . . . 10
3.6.1 投影原理分析 . . . 10
3.6.2 实验设置 . . . 11
3.6.3 四种视角透视的区别分析 . . . 11
3.6.4 代码要点展示 . . . 12
3.6.5 实验结果对比 . . . 12
3.6.6 结论分析 . . . 13

4 三维字母绘制 **13**
4.1 立体字母设计 . . . 13
4.2 核心实现代码 . . . 13
4.3 代码使用说明 . . . 14
4.3.1 键盘控制说明 . . . 14
4.4 代码亮点介绍 . . . 14
4.4.1 1. 自动化立体建模算法 . . . 14
4.4.2 2. 多模式旋转系统 . . . 15
4.4.3 3. 颜色区分 . . . 15
4.4.4 4. 用户体验优化 . . . 15
4.5 结果展示 . . . 15
4.5.1 X 轴旋转效果 . . . 16
4.5.2 Y 轴旋转效果 . . . 16
4.5.3 Z 轴旋转效果 . . . 17
4.5.4 旋转对比总结 . . . 17

5 总结与体会 **17**
5.1 实验过程中的困难与解决方案 . . . 17
5.1.1 1. OpenGL 环境配置问题 . . . 17

5.1.2 2. 字母顶点坐标设计困难 18
5.1.3 3. 三维立体建模的侧面生成问题 18
5.1.4 4. 二维视角图参数调节困难 18
5.2 实验收获与反思 19
5.2.1 技术能力提升 19
5.2.2 理论知识深化 19
5.2.3 对计算机图形学的认识 19
5.3 实验相关的代码文件 19
5.4 参考资料 20

1 实验概述

1.1 实验目的

本实验旨在通过 OpenGL 环境搭建和基本图形绘制，掌握计算机图形学中的基础概念和实现方法。具体包括：

1. 掌握 OpenGL 开发环境的搭建过程
2. 理解 OpenGL 基本图元的使用方法
3. 学会在二维平面绘制复杂图形
4. 掌握三维图形绘制和变换操作
5. 比较不同投影方式和绘制方法的差异

1.2 实验内容

1. 环境搭建：

(1) 要求在实验报告中详细描述环境搭建过程、中间遇到的困难及问题的解决方案。

(2) 建议超算习堂模板中的环境配置教程完成实验环境搭建和调试，请尽量按照教程中的环节搭建，以方便统一作业批改环境。

(3) 若使用其他方法搭建环境并实现作业内容，请给出详细的程序运行说明文档。

2. 绘制平面姓名首字母：

(1) 基本要求：

- 在二维画布（x 平面）上，使用基本图元（三角形），以原点为绘制中心，绘制自己姓名首字母（绘制结果参考图 1）。

(2) 字母选取要求：

- 当名字为两字时，需补一个顺位字母，例如姓名首字母为 LS，则需要绘制 LST 三个字母；
- 当名字大于三字时，取前三个字的首字母即可；
- 当名字中存在相同首字母时，将第二个相同的字母替换为顺位字母，例如姓名首字母为 LL，则需要绘制 LMN；
- 当姓名首字母中重复的字母为 Z 时，取循环顺位字母 A。

(3) 讨论：

- 比较 GL_TRIANGLES, GL_TRIANGLE_STRIP, GL_QUADS, GL_QUAD_STRIP 的绘制开销（需要的 glVertex 调用次数）；
- 比较以下两个视角下，Orthogonal 及 Perspective 投影方式产生的图像：

  (a) 从（0,0,d）看向原点 (0, 0, 0)；

  (b) 从 (0, 0.5*d, d) 看向原点 (0, 0, 0)。

注：可使用 (0, 1, 0) 作为视角的上方向（up）。

1. 图 1 二维字母绘制图例见下图：

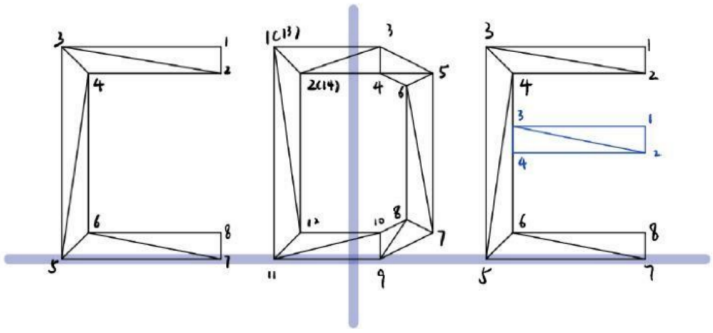


图 1: 二维字母绘制图例

3. 绘制立体姓氏首字母：

(1) 基本要求：

- 在三维空间内，以原点为绘制中心，绘制立体的姓氏首字母。

(2) 绘制要求：

- 若姓氏大于一字，取第一字的首字母；
- 要求在 xoy 平面绘制字母的 2D 样式，并在 z 轴上增加字母厚度以达到立体效果，此处厚度不做要求、可自定义。

(3) 实现内容：

- 分别实现并比较字母绕 x、y、z 轴的旋转；
- 旋转的方式不作限制，若有多种旋转方式，可对比旋转方式，比如以 x 轴为旋转轴顺时针旋转和以 x 轴为旋转轴逆时针旋转的对比。

2 环境搭建

1. 安装 Visual Studio 和 CMake

Visual Studio（下称 VS）一般选择 2022 版本，下载链接见`https://visualstudio.microsoft.com/zh-hans/`。安装组件时，查阅资料安装了以下工作负载和单个组件：

- 使用 C++ 的游戏开发：计算机图形学开发必备，包含创建 3D 图形、游戏引擎等所需的核心 C++ 工具。
- 使用 C++ 的桌面开发：提供额外工具和示例，使用 Clang/LLVM 工具集构建时必需。
- .NET 桌面开发：如项目涉及.NET 桌面应用可选安装。
- C++ 分析工具：有助于代码质量和性能优化。
- C++ AddressSanitizer：用于检测内存错误（可选）。

- Windows 10 或 11 SDK（10.0.18362 或更高版本）：提供 Windows 系统级 API 和功能支持。
- HLSL 工具：用于编写和调试着色器（如涉及 DirectX 开发）。
- 面向 Unreal 引擎的 IDE 支持：如使用 Unreal 引擎开发需勾选。

安装完成截图如下：



图 2: Visual Studio 安装成功截图

CMake 下载见`https://cmake.org/download/`，安装的版本为 cmake-4.2.0-rc1。安装成功截图如下：

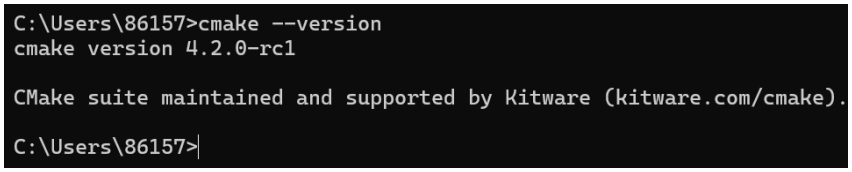


图 3: CMake 安装成功截图

2. 写 CMakeLists.txt

选定一个空文件夹作为项目根目录，在该文件夹中新建 CMakeLists.txt 文件，并将文档提供内容复制进去。

3. 写 cpp 文件

在项目根目录创建 src 文件夹，在 src 文件夹中创建 main.cpp 文件，填入文档所给内容，用于测试环境是否配置成功。

4. CMake 构建

在项目文件夹根目录（CMakeLists.txt 所在目录）打开终端（如 CMD、Powershell 等），依次输入如下命令：

```
mkdir build
cd build
cmake ..
cmake --build .
```

构建结果截图如下：

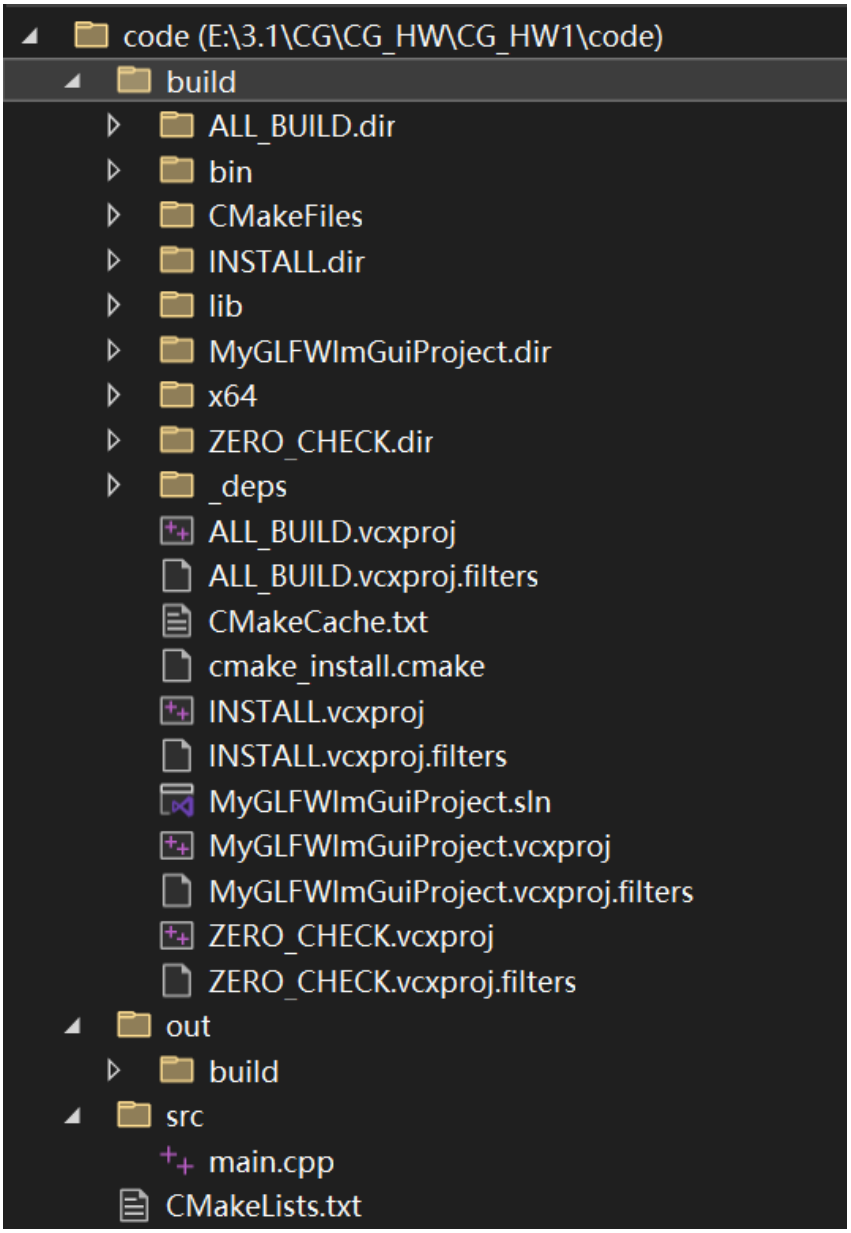


图 4: CMake 构建成功截图

5. 用 Visual Studio 打开项目文件夹

进入 Visual Studio 界面，点击“打开本地文件夹”，选择 CMakeLists.txt 所在的文件夹。稍等片刻，VS 会自动根据 CMakeLists.txt 配置项目。运行测试代码，在 VS 左侧窗口中，双击 src 中的 main.cpp 打开并运行。

运行成功截图如下：

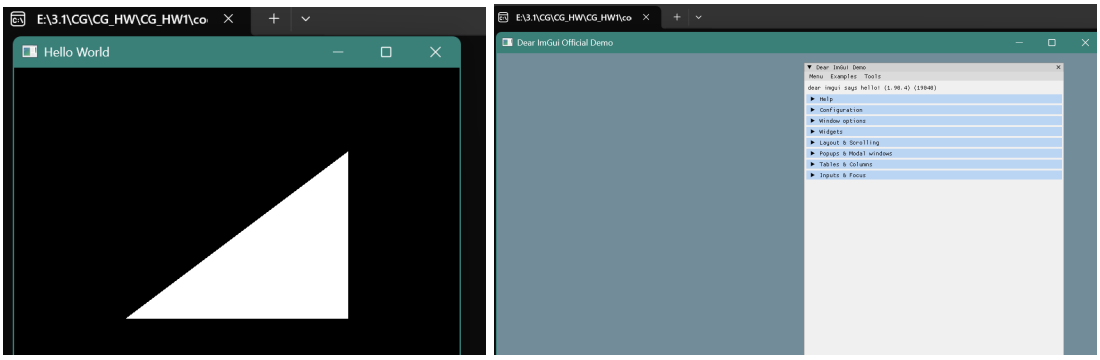


图 5: 测试代码运行成功截图

3　二维字母绘制

3.1　字母选取规则

根据作业要求，我的姓名为” 马福泉”，首字母为 MFQ。由于是三个字，直接取前三个字的首字母即可。

3.2　绘制思路

使用 OpenGL 的三角形图元（GL_TRIANGLES）来构建字母。每个字母都通过多个三角形组合而成：

- 字母 M：由 8 个三角形组成
- 字母 F：由 6 个三角形组成
- 字母 Q：由 11 个三角形组成

3.3　核心代码实现

```
const int letterM[] = {
    0, 0, 0, 0, 7, 0, 1, 0, 0,
    1, 0, 0, 0, 7, 0, 1, 5, 0,
    0, 7, 0, 1, 5, 0, 3, 4, 0,
    1, 5, 0, 3, 4, 0, 3, 3, 0,
    3, 4, 0, 3, 3, 0, 5, 5, 0,
    3, 4, 0, 5, 5, 0, 6, 7, 0,
    5, 0, 0, 5, 5, 0, 6, 7, 0,
    5, 0, 0, 6, 7, 0, 6, 0, 0
};

const int letterF[] = {
    8, 0, 0, 8, 6, 0, 9, 0, 0,
    9, 0, 0, 8, 6, 0, 9, 6, 0,
    8, 6, 0, 13, 6, 0, 8, 7, 0,
    8, 7, 0, 13, 6, 0, 13, 7, 0,
    9, 3, 0, 12, 3, 0, 9, 4, 0,
```

```
    9, 4, 0, 12, 3, 0, 12, 4, 0
};

const int letterQ[] = {
    20, 0, 0, 19, 1, 0, 21, 1, 0,
    16, 1, 0, 21, 1, 0, 20, 2, 0,
    15, 2, 0, 16, 1, 0, 20, 2, 0,
    15, 2, 0, 17, 2, 0, 15, 4, 0,
    20, 2, 0, 18, 2, 0, 20, 4, 0,
    16, 3, 0, 15, 4, 0, 16, 5, 0,
    19, 3, 0, 20, 4, 0, 19, 5, 0,
    15, 4, 0, 15, 6, 0, 17, 6, 0,
    20, 4, 0, 20, 6, 0, 18, 6, 0,
    15, 6, 0, 20, 6, 0, 16, 7, 0,
    16, 7, 0, 19, 7, 0, 20, 6, 0
};

// 绘制函数示例
void drawLetter(const int* vertices, int triangleCount) {
    glBegin(GL\textunderscore TRIANGLES);
    for (int i = 0; i < triangleCount * 9; i += 3) {
        glVertex3i(vertices[i], vertices[i + 1], vertices[i + 2]);
    }
    glEnd();
}

// 使用方法:
// drawLetter(letterM, 8);
// drawLetter(letterF, 6);
// drawLetter(letterQ, 11);
```

Listing 1: MFQ 字母顶点定义与绘制（纵坐标为 0）

3.4 绘制结果

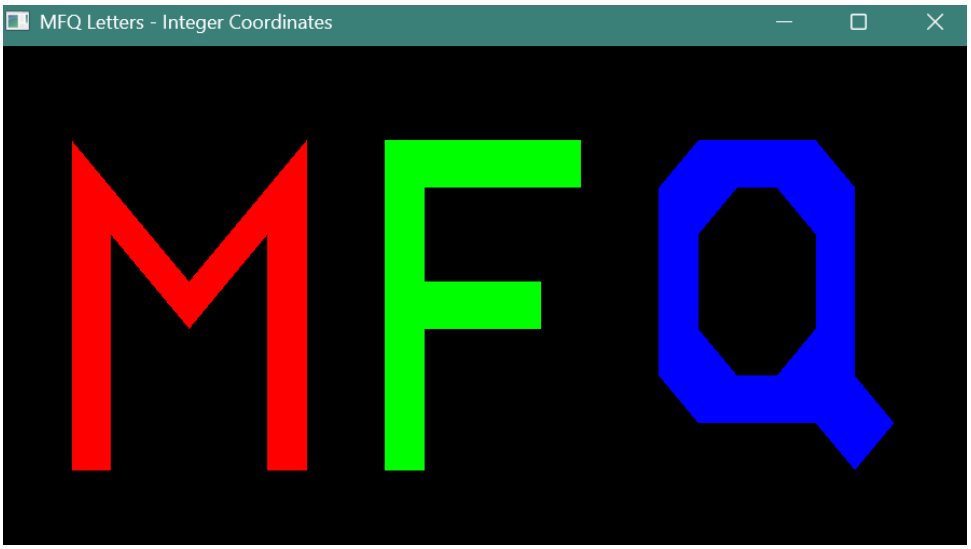


图 6: 二维字母 MFQ 绘制结果

3.5 讨论一：不同绘制方式的开销比较

在绘制字母 M、F、Q 时，我们可以使用不同的 OpenGL 图元类型。下面分析各种图元类型的特点及其绘制开销：

3.5.1 图元类型分析

- **GL_TRIANGLES**：独立三角形模式，每 3 个顶点构成一个三角形，顶点不共享。这是最基础的绘制方式，灵活性高但顶点数据冗余较多。
- **GL_TRIANGLE_STRIP**：三角形条带模式，除前两个顶点外，每增加一个顶点就形成一个新三角形。相邻三角形共享边，大大减少了顶点数量。
- **GL_QUADS**：独立四边形模式，每 4 个顶点构成一个四边形。适合绘制矩形区域，但现代 OpenGL 已不推荐使用。
- **GL_QUAD_STRIP**：四边形条带模式，除前两个顶点外，每增加 2 个顶点就形成一个新四边形。相邻四边形共享边。

3.5.2 具体开销计算

M、F、Q 的设计图如下：

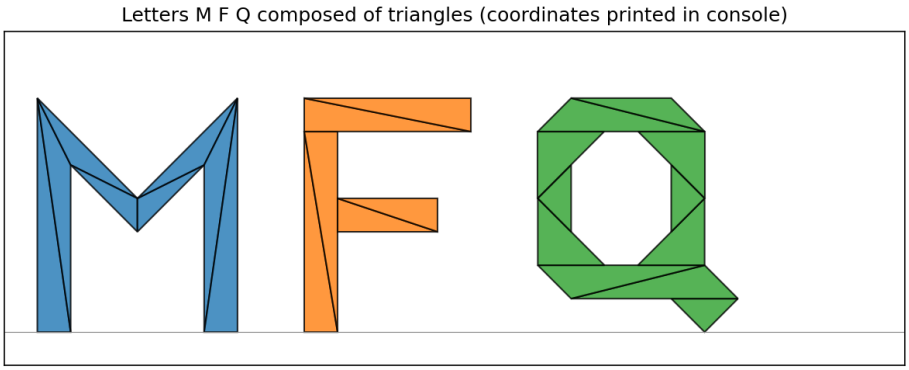


图 7: M、F、Q 的设计图

根据字母 M、F、Q 的设计图，各图元类型的绘制开销如下：

表 1: 不同图元类型的绘制开销比较

绘制方式	字母分解数量	总顶点调用次数	效率排名
GL_TRIANGLES	M(8)+F(6)+Q(11)=25 个三角形	$25 \times 3 = 75$	4
GL_TRIANGLE_STRIP	M(10)+F(10)+Q(18) 个顶点	$10 + 10 + 18 = 38$	2
GL_QUADS	M(4)+F(3)+Q(9)=16 个四边形	$16 \times 4 = 64$	3
GL_QUAD_STRIP	M(10)+F(10)+Q(16) 个顶点	$10 + 10 + 16 = 36$	1

3.5.3　结果分析

从计算结果可以看出：

1. **GL_QUAD_STRIP 效率最高**：仅需 36 次 glVertex 调用，因为相邻四边形共享边，大大减少了顶点重复。
2. **GL_TRIANGLE_STRIP 次之**：需要 38 次 glVertex 调用，三角形条带模式有效减少了顶点冗余。
3. **GL_QUADS 排第三**：需要 64 次 glVertex 调用，虽然四边形更适合描述矩形字母结构，但缺乏顶点共享机制。
4. **GL_TRIANGLES 效率最低**：需要 75 次 glVertex 调用，每个三角形都需要完整的 3 个顶点，顶点重复最多。

注意：虽然 GL_QUAD 相关的图元在某些场景下效率较高，但在现代 OpenGL（3.0+）中已被弃用，建议使用三角形图元配合索引缓冲区来实现高效绘制。

3.6　讨论二：投影方式比较

3.6.1　投影原理分析

在计算机图形学中，投影变换是将三维场景映射到二维屏幕的关键步骤。主要有两种投影方式：

- **正交投影（Orthogonal Projection）**：平行投影，物体的大小不随距离变化而改变，保持平行线的平行性。适用于工程制图、建筑图纸等需要精确尺寸的场合。

- **透视投影（Perspective Projection）**：中心投影，模拟人眼观察效果，远处的物体看起来更小，产生景深效果。符合人类视觉习惯，常用于游戏和动画渲染。

3.6.2 实验设置

本实验比较了以下四种视角组合下的投影效果：

1. **视角 1 + 正交投影**：从 (0,0,d) 看向原点 (0,0,0)，使用正交投影
2. **视角 1 + 透视投影**：从 (0,0,d) 看向原点 (0,0,0)，使用透视投影
3. **视角 2 + 正交投影**：从 (0,0.5*d,d) 看向原点 (0,0,0)，使用正交投影
4. **视角 2 + 透视投影**：从 (0,0.5*d,d) 看向原点 (0,0,0)，使用透视投影

其中，d=5.0，上方向向量为 (0,1,0)。

3.6.3 四种视角透视的区别分析

1. 视角 1 正交投影

- 观察角度：正视角度（沿 z 轴负方向观察）
- 投影特点：字母保持原始比例，无透视变形
- 视觉效果：类似于工程图纸，所有平行线保持平行
- 适用场景：技术制图、精确测量

2. 视角 1 透视投影

- 观察角度：正视角度（沿 z 轴负方向观察）
- 投影特点：字母中心区域与边缘区域有轻微的透视差异
- 视觉效果：更加真实自然，符合人眼观察习惯
- 适用场景：游戏渲染、虚拟现实

3. 视角 2 正交投影

- 观察角度：斜视角度（从上方偏移位置观察）
- 投影特点：字母形状发生倾斜，但保持平行投影特性
- 视觉效果：类似于等轴测图，可同时观察到多个面
- 适用场景：建筑设计、机械制图的轴测图

4. 视角 2 透视投影

- 观察角度：斜视角度（从上方偏移位置观察）
- 投影特点：字母既有倾斜又有透视变形，最接近真实观察效果
- 视觉效果：最强的三维立体感和空间深度感
- 适用场景：3D 游戏、电影特效、虚拟现实应用

3.6.4 代码要点展示

在 OpenGL 中实现不同投影方式的关键代码片段：

```
// 正交投影设置
void setOrthogonalProjection() {
    glMatrixMode(GL\textunderscore PROJECTION);
    glLoadIdentity();
    glOrtho(-2.0, 2.0, -2.0, 2.0, 1.0, 10.0);
}

// 透视投影设置
void setPerspectiveProjection() {
    glMatrixMode(GL\textunderscore PROJECTION);
    glLoadIdentity();
    gluPerspective(45.0, 1.0, 1.0, 10.0);
}

// 视角设置
void setViewpoint(int viewType) {
    glMatrixMode(GL\textunderscore MODELVIEW);
    glLoadIdentity();

    if(viewType == 1) {
        // 视角1: 从(0,0,5)看向原点
        gluLookAt(0.0, 0.0, 5.0,     // 眼睛位置
                  0.0, 0.0, 0.0,     // 目标位置
                  0.0, 1.0, 0.0);    // 上方向
    } else {
        // 视角2: 从(0,2.5,5)看向原点
        gluLookAt(0.0, 2.5, 5.0,     // 眼睛位置
                  0.0, 0.0, 0.0,     // 目标位置
                  0.0, 1.0, 0.0);    // 上方向
    }
}
```

Listing 2: 投影矩阵设置

3.6.5 实验结果对比

投影方式比较的实验结果如下图所示：

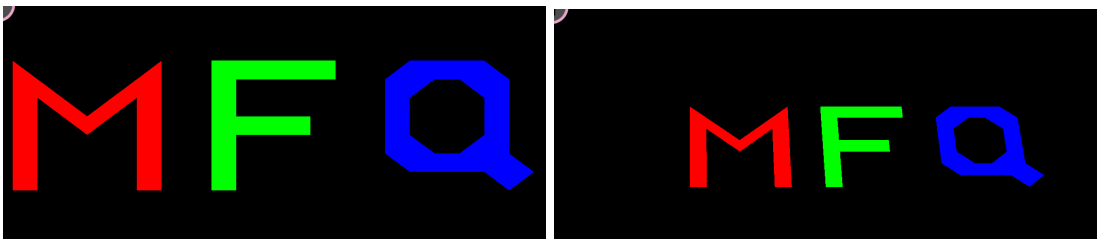


图 8: 视角 1 下的正交投影（左）和透视投影（右）比较

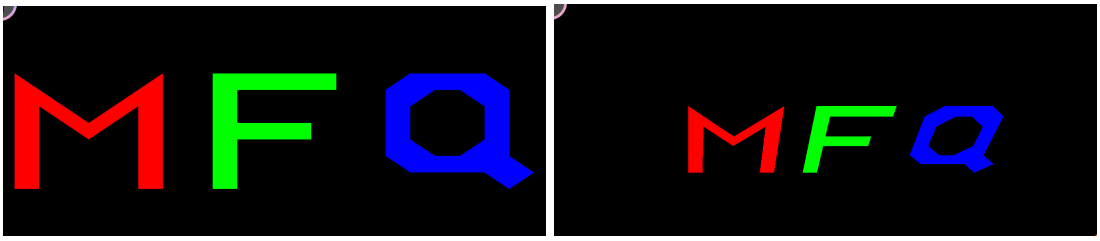


图 9: 视角 2 下的正交投影（左）和透视投影（右）比较

3.6.6 结论分析

通过对比实验可以得出：

1. **正交投影**适合需要精确尺寸的技术应用，但缺乏真实感
2. **透视投影**提供更真实的视觉效果，适合交互式图形应用
3. **视角变化**对投影效果有显著影响，斜视角度能够提供更丰富的空间信息
4. **投影方式选择**应根据具体应用需求确定：工程制图选择正交投影，游戏渲染选择透视投影

4 三维字母绘制

4.1 立体字母设计

根据要求，绘制姓氏” 马” 的首字母”M” 的立体版本。设计思路：

1. 在 xoy 平面绘制字母 M 的 2D 轮廓
2. 沿 z 轴方向拉伸，增加厚度
3. 添加前后面和侧面，形成完整的立体效果

4.2 核心实现代码

```
// 3D顶点结构
struct Vertex {
    float x, y, z;
    Vertex(float x, float y, float z) : x(x), y(y), z(z) {}
};

// 三角形结构
struct Triangle {
    Vertex v1, v2, v3;
    Triangle(Vertex v1, Vertex v2, Vertex v3) : v1(v1), v2(v2), v3(v3) {}
};

// 立体字母M的三角形集合
std::vector<Triangle> letterM3D;
const float THICKNESS = 2.0f;  // 厚度参数
```

Listing 3: 3D 字母 M 数据结构定义

```
// 旋转参数
float angleX = 0.0f, angleY = 0.0f, angleZ = 0.0f;
int rotationAxis = 0; // 0:x, 1:y, 2:z, 3:组合旋转

// 旋转更新逻辑
if (autoRotate) {
    switch (rotationAxis) {
        case 0: angleX += rotationSpeed; break;          // X轴旋转
        case 1: angleY += rotationSpeed; break;          // Y轴旋转
        case 2: angleZ += rotationSpeed; break;          // Z轴旋转
        case 3: // 组合旋转
            angleX += rotationSpeed;
            angleY += rotationSpeed * 0.7f;
            angleZ += rotationSpeed * 0.3f;
            break;
    }
}

// 应用旋转变换
glRotatef(angleX, 1.0f, 0.0f, 0.0f);
glRotatef(angleY, 0.0f, 1.0f, 0.0f);
glRotatef(angleZ, 0.0f, 0.0f, 1.0f);
```

Listing 4: 多模式旋转系统

4.3 代码使用说明

4.3.1 键盘控制说明

- **X/Y/Z 键**：分别启动绕 X、Y、Z 轴的自动旋转
- **A 键**：启动 XYZ 组合旋转模式
- **R 键**：重置所有旋转角度到初始状态
- **空格键**：切换自动旋转和手动旋转模式
- **+/-键**：动态调整旋转速度
- **方向键**：手动模式下控制 X、Y 轴旋转
- **Q/E 键**：手动模式下控制 Z 轴旋转

4.4 代码亮点介绍

4.4.1 1. 自动化立体建模算法

实现了从 2D 轮廓到 3D 立体的转换：

- **前后表面生成**：自动从 2D 顶点生成前表面，并通过 Z 轴偏移创建后表面
- **智能侧面构建**：自动识别 2D 轮廓的边缘，生成连接前后表面的侧面三角形

- **法线方向处理**：后表面顶点顺序自动反转，确保正确的面法线方向

4.4.2　2. 多模式旋转系统

实现了多种旋转模式：

- **单轴旋转**：绕 X、Y、Z 轴的独立旋转，便于观察各轴旋转效果
- **组合旋转**：三轴同时旋转但速度不同，创造复杂的 3D 运动轨迹
- **自动/手动切换**：支持自动播放和手动精确控制两种模式
- **动态速度调节**：实时调整旋转速度，满足不同观察需求

4.4.3　3. 颜色区分

基于几何特征的智能着色系统：

- **自动面识别**：根据顶点 Z 坐标自动判断前表面、后表面和侧面
- **差异化着色**：前表面（红色）、后表面（蓝色）、侧面（绿色）
- **立体感增强**：不同颜色有助于用户更好地理解 3D 结构

4.4.4　4. 用户体验优化

注重交互体验的细节设计：

- **防抖处理**：空格键切换时避免连续触发
- **边界保护**：旋转速度调节有下限保护
- **实时反馈**：控制台输出详细的操作说明
- **状态可视化**：不同旋转模式有明显的视觉区别

4.5　结果展示

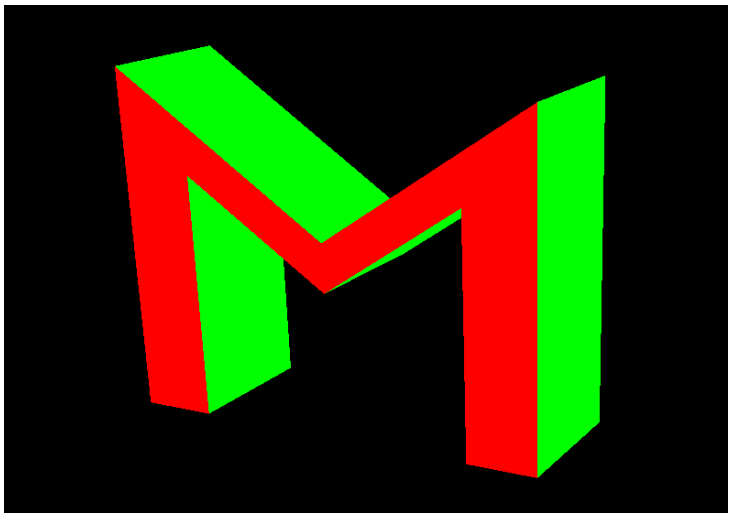


图 10: 静态 3D 字母 M 展示

4.5.1 X 轴旋转效果

字母 M 绕 X 轴旋转的关键帧序列展示旋转过程:

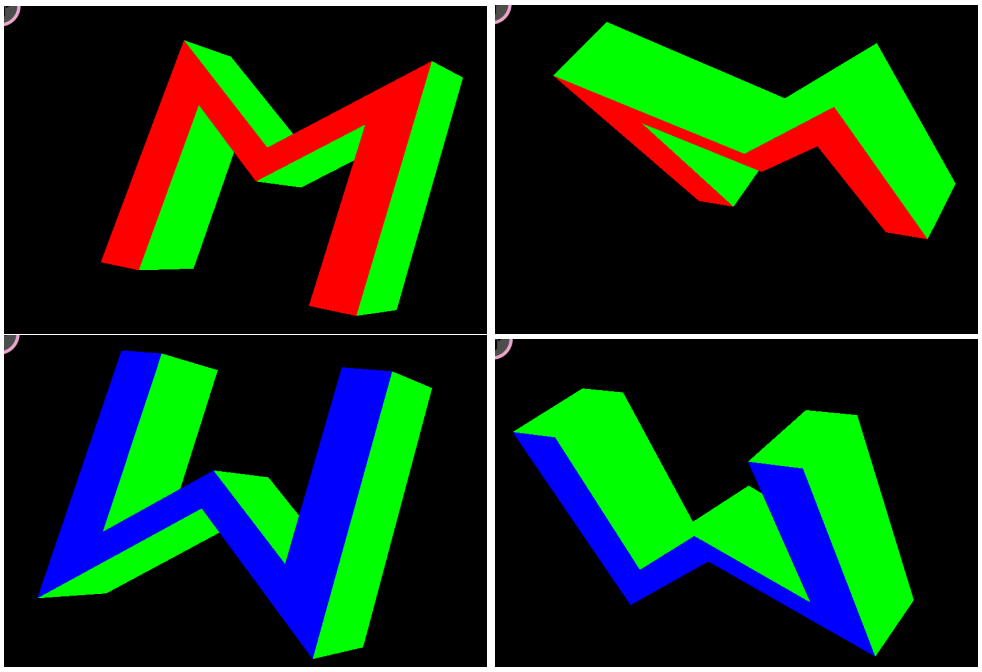


图 11: 绕 X 轴旋转关键帧

4.5.2 Y 轴旋转效果

字母 M 绕 Y 轴旋转的关键帧序列展示旋转过程:

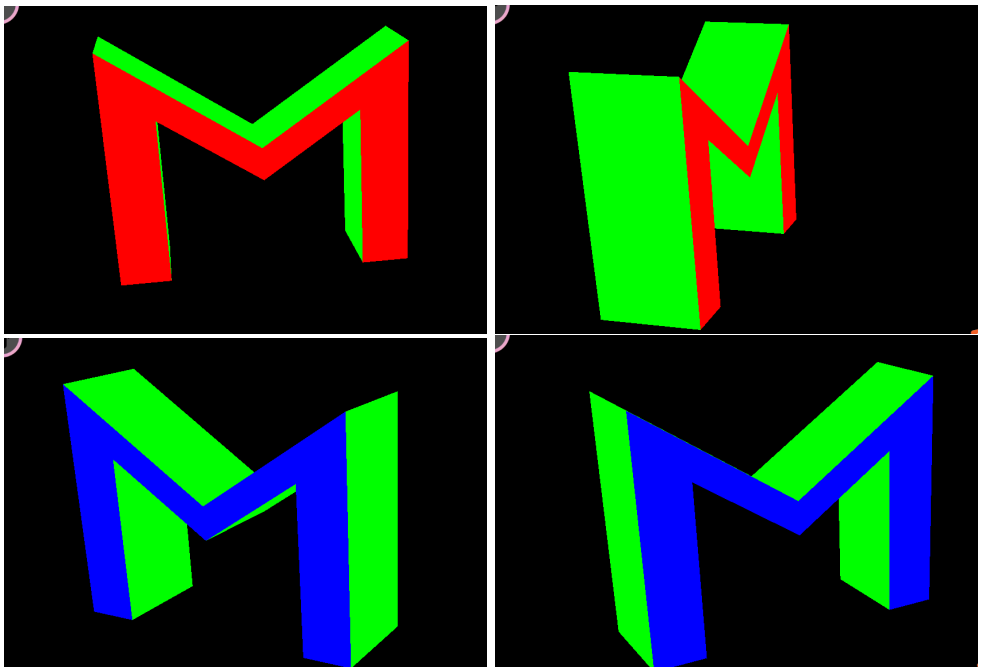


图 12: 绕 Y 轴旋转关键帧

4.5.3 Z 轴旋转效果

字母 M 绕 Z 轴旋转的关键帧序列展示旋转过程:

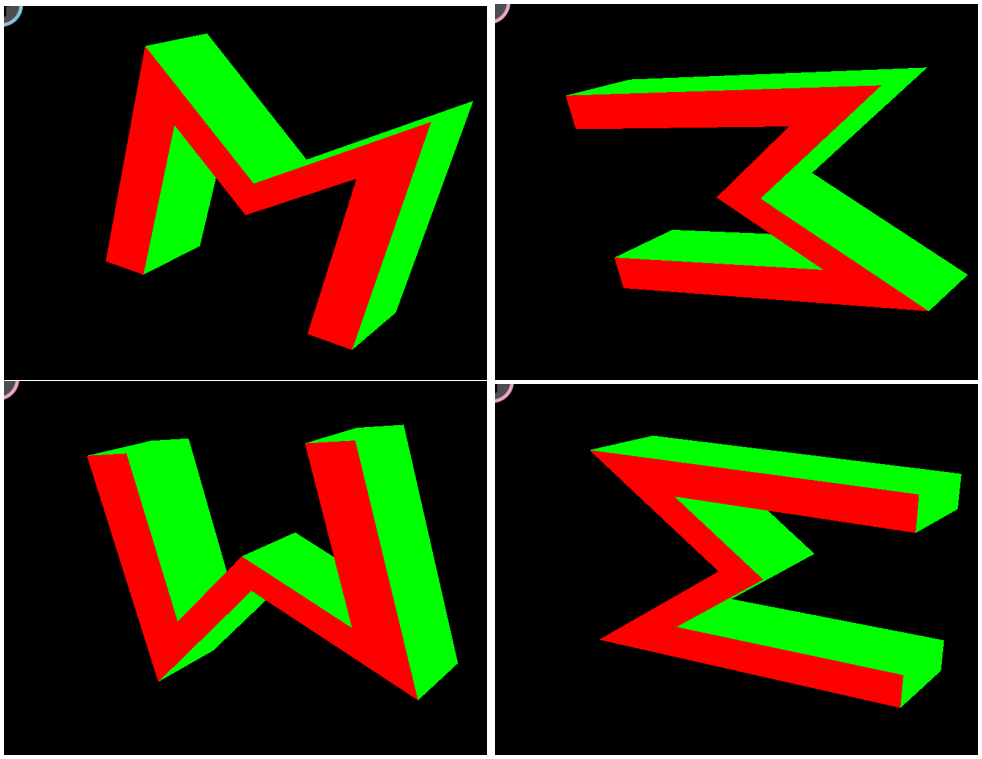


图 13: 绕 Z 轴旋转关键帧

4.5.4 旋转对比总结

通过对比三个方向的旋转效果, 可以得出以下结论:

1. **X 轴旋转**适合观察字母的上下结构和垂直方向的厚度变化
2. **Y 轴旋转**最能体现字母的立体感和左右侧面的厚度
3. **Z 轴旋转**保持字母正面朝向, 适合展示字母在平面内的方向变化
4. 三种旋转方式各有特点, 组合使用可以全方位展示 3D 字母的立体结构

5 总结与体会

5.1 实验过程中的困难与解决方案

以下是本次实验的实施过程遇到的主要困难及其解决方案：

5.1.1 1. OpenGL 环境配置问题

困难描述：
初次配置 OpenGL 开发环境时，遇到了 CMake 无法找到 GLFW 和 GLAD 库的问题。编译时出现” 找不到 glfw3.lib” 和头文件路径错误等问题。
解决方案：

- 仔细阅读实验指导书和 CMakeLists.txt 配置文件，理解第三方库的链接方式
- 将 GLAD 的头文件和源文件正确放置在项目目录中
- 使用绝对路径进行调试，确认路径后再改为相对路径

5.1.2 2. 字母顶点坐标设计困难

困难描述：

在设计字母 M、F、Q 的顶点坐标时，难以准确控制字母的形状和比例。特别是字母 Q 的圆形部分，使用三角形近似时效果不理想，出现明显的棱角。

解决方案：

- 先在方格纸上手绘字母轮廓，确定关键顶点的整数坐标
- 对于字母 Q 的圆形部分，增加三角形数量以提高近似精度
- 使用数组存储顶点数据，便于调试和修改
- 编写辅助函数可视化顶点序号，方便检查三角形的连接关系

5.1.3 3. 三维立体建模的侧面生成问题

困难描述：

在从 2D 字母生成 3D 立体字母时，前后表面的生成比较简单，但侧面的自动生成遇到困难。需要识别 2D 轮廓的边缘，并正确连接前后表面的对应顶点，且要保证三角形的法线方向一致。

解决方案：

- 分析 2D 三角形的边缘特征，提取轮廓边
- 为每条边缘创建两个三角形连接前后表面
- 使用右手法则确保法线方向朝外
- 通过颜色区分前面、后面和侧面，便于调试

5.1.4 4. 二维视角图参数调节困难

困难描述：

在绘制二维字母的不同视角投影图时，发现字母经常显示不完整或比例失衡，需要不断调整视野大小、投影参数和观察点位置。尤其是在切换正交与透视投影、不同视角时，参数设置稍有不当就会导致图像超出窗口或变形。

解决方案：

- 参考 OpenGL 官方文档，理解 glOrtho 和 gluPerspective 各参数的实际含义
- 结合字母实际尺寸，合理设置视野范围和裁剪面
- 多次实验不同参数，记录效果，最终选取合适的参数组合

5.2 实验收获与反思

通过本次基于 OpenGL 的字母绘制实验，我在计算机图形学的理论知识和实践技能方面都获得了提升。以下是主要收获和反思：

5.2.1 技术能力提升

1. **图形 API 掌握**：深入理解了 OpenGL 的工作原理，掌握了基本图元、顶点缓冲、变换矩阵等核心概念。能够熟练使用 glBegin/glEnd 进行图形绘制，理解了现代 OpenGL 与传统 OpenGL 的区别。

2. **三维空间思维**：通过三维字母的建模和旋转实验，培养了三维空间想象能力。能够在脑海中构建物体的三维模型，理解不同视角下物体的投影效果。

3. **算法设计能力**：实现了从 2D 到 3D 的自动建模算法，提升了几何算法的设计能力。学会了如何将复杂问题分解为简单子问题，通过模块化设计提高代码的可维护性。

4. **调试与优化**：积累了图形程序调试的经验，学会使用颜色区分、辅助线绘制等方法定位问题。理解了顶点数据优化对性能的影响，掌握了不同绘制模式的性能差异。

5.2.2 理论知识深化

1. **投影变换理论**：通过实际对比正交投影和透视投影的效果，深刻理解了两种投影方式的数学原理和应用场景。认识到投影方式的选择应根据具体需求确定。

2. **坐标系统**：系统掌握了世界坐标系、观察坐标系、裁剪坐标系等概念，理解了从模型空间到屏幕空间的完整变换流程。

3. **图形渲染管线**：了解了 OpenGL 渲染管线的基本流程，理解了顶点处理、图元装配、光栅化、片段处理等阶段的作用。

4. **性能优化原理**：通过比较不同图元类型的绘制开销，理解了顶点共享、索引缓冲等优化技术的重要性。

5.2.3 对计算机图形学的认识

通过本次实验，我认识到计算机图形学是一门理论与实践紧密结合的学科，不仅需要扎实的数学基础（线性代数、几何学），还需要良好的编程能力和空间想象力。本次实验不仅让我掌握了 OpenGL 的基本使用方法，更重要的是培养了图形学思维方式和问题解决能力。这将成为我继续深入学习计算机图形学的坚实基础。

5.3 实验相关的代码文件

main_2d.cpp: 二维字母绘制代码文件 main_3d.cpp: 三维字母绘制代码文件

注意

运行时需要将对应的代码文件重命名为 `main.cpp`

5.4 参考资料

1. OpenGL 官方文档：`https://www.opengl.org/documentation/`
2. GLFW 官网：`https://www.glfw.org/`
3. LearnOpenGL 教程：`https://learnopengl.com/`
4. 《计算机图形学》教材相关章节